

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»
Факультет Программной инженерии и компьютерной техники

Лабораторная работа №2
«Использование шаблонов проектирования»
по дисциплине
«Архитектура программных систем»

Выполнил студент группы Р3307:
Садовой Григорий Владимирович

Преподаватель:
Перл Иван Андреевич

г. Санкт-Петербург
2025

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	2
1. ЗАДАНИЕ	3
2. Bridge (GoF)	4
2.1. Описание	4
2.2. Сценарии использования	4
2.2.1. Воспроизведение аудиоконтента на разных платформах.....	4
2.2.2. Отрисовка графики на разных API и платформах.....	4
2.3. Общие ограничения Bridge	5
3. Singleton (GoF)	6
3.1. Описание	6
3.2. Сценарии использования	6
3.2.1. Единый диспетчер учёта времени в пропускной системе.....	6
3.2.2. Единый диспетчер состояния железнодорожной инфраструктуры	6
3.3. Общие ограничения Singleton	7
4. Creator (GRASP)	8
4.1. Описание	8
4.2. Сценарии использования	8
4.2.1. Создание объектов заказа в интернет-магазине (Amazon-подобное приложение)	8
4.2.2. Формирование нарушения и штрафа на основе события распознавания (система социального рейтинга).....	8
4.3. Общие ограничения Creator	9
5. Information Expert (GRASP)	10
5.1. Описание	10
5.2. Сценарии использования	10
5.2.1. Расчёт стоимости церковных услуг в приложении удаленно	10
5.2.2. Расчёт запаса хода в бортовой системе автомобиля	10
5.3. Общие ограничения Information Expert.....	11
6. ВЫВОД	12

1. ЗАДАНИЕ

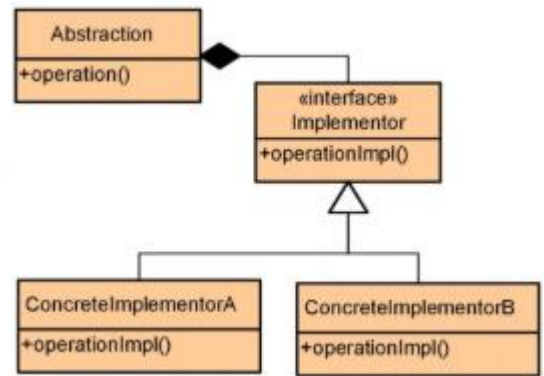
Из списка шаблонов проектирования GoF и GRASP выбрать 3-4 шаблона и для каждого из них придумать 2-3 сценария, для решения которых могут применены выбранные шаблоны.

Сделать предположение о возможных ограничениях, к которым можем привести использование шаблона в каждом описанном случае. Обязательно выбрать шаблоны из обоих списков.

2. Bridge (GoF)

2.1. Описание

Bridge (Мост)— **структурный** паттерн, который разделяет абстракцию и её реализацию на две независимые иерархии и связывает их через композицию (абстракция хранит ссылку на объект-реализацию и делегирует ему часть работы). Такой подход позволяет изменять и расширять абстракции (например, разные виды поведения или сценарии использования) независимо от конкретных реализаций (например, платформ, движков, провайдеров), не создавая множества классов на каждую комбинацию вариантов. Паттерн особенно полезен, когда в системе есть два и более “измерения” изменчивости и необходимо избежать разрастания кода, сохраняя низкую связанность и возможность развивать части системы отдельно друг от друга.



2.2. Сценарии использования

2.2.1. Воспроизведение аудиоконтента на разных платформах

В сервисе потоковой музыки необходимо воспроизводить разные типы контента (музыкальные треки, подкасты, аудиокниги) на разных платформах (Android, iOS, Web). Абстракцией выступают плееры (MusicPlayer, PodcastPlayer, AudiobookPlayer), реализующие продуктовую логику воспроизведения, а реализацией платформенные движки (AndroidAudioEngine, iOSAudioEngine, WebAudioEngine), инкапсулирующие детали работы со звуком. Паттерн Bridge позволяет развивать типы плееров и движки воспроизведения независимо друг от друга и избегать создания классов для каждой комбинации «тип контента × платформа».

Ограничения:

- Потеря платформенной специфики: общий интерфейс PlaybackEngine может не поддерживать все возможности конкретных платформ (например, особые режимы аудиосессий или оптимизации буферизации), из-за чего часть функций плееров придётся упрощать или реализовывать обходным способом.
- Несовместимость ожиданий плеера и движка: разные типы плееров (например, PodcastPlayer с изменяемой скоростью) могут требовать функций, которые реализованы не во всех движках, что приведёт к проверкам возможностей и усложнению логики.

2.2.2. Отрисовка графики на разных API и платформах

В игре вида The Witcher требуется поддерживать разные графические API и платформы (например, DirectX на Windows и Vulkan/Metal на других платформах), при этом игровые объекты и эффекты должны обрабатываться одинаково на уровне игровой логики. Абстракцией выступают компоненты рендеринга сцены (CharacterRenderer,

EnvironmentRenderer, EffectsRenderer), описывающие «что рисовать», а реализацией выступает графический бэкенд (DirectXBackend, VulkanBackend, MetalBackend), который реализует как рисовать: командные буферы, шейдеры, ресурсы. Bridge позволяет независимо добавлять новые виды рендера и развивать/заменять графические бэкенды без роста числа классов по всем комбинациям..

Ограничения:

- Ограничение общего контракта: интерфейс графического бэкенда часто приходится делать усреднённым, из-за чего специфичные возможности отдельных API используются хуже или требуют специальных расширений.
- Рост сложности интеграции и отладки: ошибки рендера могут возникать на границе «Renderer ↔ Backend», что увеличивает трудоёмкость диагностики и требует строгих контрактов и хорошего логирования.

2.3. Общие ограничения Bridge

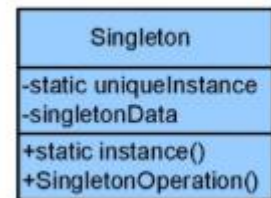
1. **Избыточная сложность:** Паттерн Bridge вводит две иерархии классов вместо одной, что увеличивает количество сущностей и связей между ними. В системах с небольшим числом вариантов это может привести к избыточной архитектурной сложности без существенной практической выгоды.
2. **Сложность проектирования интерфейса реализации:** Интерфейс реализации должен быть достаточно универсальным для поддержки всех абстракций. Слишком узкий интерфейс ограничивает развитие абстракции, а слишком широкий делает конкретные реализации перегруженными и трудными для сопровождения.
3. **Усложнение отладки и сопровождения:** Увеличение длины цепочки вызовов затрудняет диагностику ошибок и требует дополнительных усилий по логированию и тестированию взаимодействия между уровнями.

3. Singleton (GoF)

3.1. Описание

Singleton (одиночка) – порождающий паттерн, гарантирующий существование ровно одного экземпляра класса и предоставляющий глобальную точку доступа к нему. Он применяется, когда в системе требуется единый объект для управления общими ресурсами или координации работы компонентов, например для конфигурации, логирования или кэширования.

Паттерн реализуется за счёт сокрытия конструктора и предоставления статического метода или свойства для получения экземпляра. Это обеспечивает централизованный доступ к состоянию, но одновременно усиливает связанность компонентов с глобальным объектом, что требует осторожного и обоснованного применения.



3.2. Сценарии использования

3.2.1. Единый диспетчер учёта времени в пропускной системе

В сервисе контроля времени, основанном на данных пропускной системы, требуется централизованно обрабатывать события входа и выхода сотрудников, рассчитывать фактически проведенное время, фиксировать опоздания и формировать агрегированные отчёты. Для этого может использоваться Singleton в виде единственного объекта TimeTrackingManager, который принимает события от считывателей, применяет единые правила расчёта и записывает результаты в хранилище. Наличие одного экземпляра обеспечивает согласованность расчётов и предотвращает расхождения в состоянии при параллельной обработке событий.

Ограничения:

- Глобальное состояние и тестируемость: Singleton создаёт скрытую зависимость для многих компонентов, усложняет модульное тестирование и может приводить к неочевидным эффектам из-за общего состояния между тестами и модулями.
- Масштабирование и конкуренция: единственный экземпляр может стать узким местом при высокой нагрузке или в распределённой архитектуре; для нескольких серверов потребуется дополнительный механизм синхронизации, иначе появится риск рассогласования данных.

3.2.2. Единый диспетчер состояния железнодорожной инфраструктуры

В системе контроля железнодорожных путей требуется централизованно отслеживать состояние инфраструктуры, включая участки путей, стрелочные переводы, сигналы и занятость перегонов по данным датчиков и систем автоматики. Для этого может использоваться Singleton в виде единственного объекта InfrastructureStateManager, который принимает телеметрию от различных источников, обновляет текущее состояние сети, выявляет аварийные ситуации и предоставляет согласованные данные другим компонентам системы, таким как модули диспетчеризации, планирования движения и оповещения. Наличие одного экземпляра гарантирует, что все подсистемы работают с

единым и согласованным представлением состояния железнодорожной сети и исключает противоречия, связанные с параллельным обновлением данных.

Ограничения:

- **Глобальное состояние и связность:** Singleton усиливает зависимость компонентов от глобального объекта состояния, что затрудняет модульное тестирование и усложняет замену или эволюцию логики управления инфраструктурой.
- **Масштабируемость и отказоустойчивость:** единственный экземпляр может стать узким местом и единой точкой отказа; в распределённой системе потребуется дополнительная синхронизация и репликация состояния, иначе возникнет риск рассогласования данных между узлами.

3.3. Общие ограничения Singleton

1. **Сложность тестирования:** Наличие единственного экземпляра и общего состояния усложняет модульное тестирование, так как тесты могут влиять друг на друга через Singleton. Для изоляции тестов требуется дополнительная логика сброса или подмены экземпляра.
2. **Трудность расширения и замены реализации:** Жёсткая привязка к конкретному классу Singleton затрудняет замену реализации или внедрение альтернативных вариантов поведения без изменения клиентского кода.
3. **Ограниченная масштабируемость:** В многопоточных и распределённых системах Singleton может стать узким местом и единой точкой отказа. Для корректной работы требуется синхронизация доступа и дополнительные механизмы согласования состояния между узлами.

4. Creator (GRASP)

4.1. Описание

Creator (Создатель) - принцип GRASP, определяющий, какому объекту следует поручить ответственность за создание экземпляров другого объекта. В соответствии с этим принципом объект должен создавать другой объект, если он агрегирует его, содержит его как часть своего состояния, активно использует его или обладает данными, необходимыми для его инициализации. Основная цель принципа Creator заключается в снижении связанности между компонентами системы и в обеспечении логически обоснованного распределения обязанностей по созданию объектов. Такое распределение повышает связность классов и делает архитектуру более понятной и устойчивой к изменениям.

4.2. Сценарии использования

4.2.1. Создание объектов заказа в интернет-магазине (Amazon-подобное приложение)

В интернет-магазине пользователь формирует корзину, добавляет товары и оформляет заказ. После подтверждения покупки необходимо создать объект Order, содержащий список позиций, цены, количество товаров, адрес доставки и выбранный способ оплаты. В соответствии с принципом Creator ответственность за создание объекта Order может быть возложена на объект ShoppingCart, так как именно он агрегирует позиции заказа, хранит данные о товарах и обладает всей информацией, необходимой для инициализации заказа. Такое решение позволяет сосредоточить логику создания заказа в объекте, который логически связан с его содержимым, и избежать передачи большого количества параметров через внешние сервисы или контроллеры.

Ограничения:

- Риск перегрузки корзины: если в объект ShoppingCart добавлять не только хранение товаров, но и сложную логику создания заказов, валидации и расчёта стоимости, он может стать перегруженным и нарушить принцип единой ответственности.
- Ограниченная гибкость: при изменении процесса оформления заказа (например, добавление новых типов заказов или альтернативных способов расчёта) может потребоваться модификация ShoppingCart, что усложняет сопровождение.

4.2.2. Формирование нарушения и штрафа на основе события распознавания (система социального рейтинга)

В системе, анализирующей видеопотоки с камер и фиксирующей нарушения правил поведения, после обнаружения инцидента необходимо создать объект ViolationRecord с данными о времени, месте, типе нарушения и личности нарушителя, а также объект Penalty с параметрами штрафа и изменением социального рейтинга. В соответствии с принципом Creator ответственность за создание ViolationRecord может быть возложена на компонент ViolationDetector, так как он агрегирует данные события и обладает всей необходимой информацией для инициализации записи. Аналогично, объект Penalty создаётся компонентом PenaltyCalculator, поскольку именно он содержит правила расчёта санкций.

Ограничения:

- Перегрузка компонентов: если ViolationDetector или PenaltyCalculator будут отвечать не только за создание объектов, но и за дополнительные обязанности, они могут стать чрезмерно сложными.
- Чувствительность к изменениям правил: при изменении политики штрафов и расчёта рейтинга потребуется модификация компонентов-создателей, что усложняет сопровождение системы.

4.3. Общие ограничения Creator

- **Риск перегрузки объектов:** Назначение ответственности за создание объектов тем же компонентам, которые агрегируют данные или активно используют создаваемые экземпляры, может привести к их перегрузке дополнительной логикой. Это нарушает принцип единой ответственности и снижает связность классов.
- **Ограниченная применимость в сложных сценариях создания:** В случаях, когда процесс создания объекта зависит от множества внешних факторов или конфигураций, применение Creator может быть недостаточным, и потребуется введение специализированных фабрик или сервисов создания.
- **Снижение гибкости при изменении процесса создания:** Если логика инициализации объектов усложняется (например, добавляются новые параметры, этапы валидации или внешние зависимости), компоненты, реализующие принцип Creator, требуют постоянных изменений, что повышает стоимость сопровождения.

5. Information Expert(GRASP)

5.1. Описание

Information Expert (Информационный эксперт) — принцип GRASP, согласно которому ответственность за выполнение операции должна назначаться тому объекту, который обладает необходимой информацией для её выполнения. Идея заключается в том, что логика должна находиться как можно ближе к тем данным, с которыми она работает. Такой подход позволяет распределять обязанности естественным образом, повышает связность классов и делает систему более понятной. Например, объект заказа должен уметь считать свою стоимость, так как он содержит позиции и цены, а объект банковского счёта должен проверять возможность списания средств, так как он знает текущий баланс и лимиты. Принцип Information Expert способствует уменьшению количества лишних зависимостей между классами и помогает избежать концентрации бизнес-логики в одном центральном компоненте.

5.2. Сценарии использования

5.2.1. Расчёт стоимости церковных услуг в приложении удаленно

В приложении, позволяющем заказывать церковные услуги (постановка свечей, записки, молебны), каждый заказ содержит список выбранных услуг, их количество и размеры пожертвований. В соответствии с принципом Information Expert ответственность за расчёт общей суммы заказа возлагается на объект ChurchOrder, так как именно он агрегирует все позиции и знает их параметры. Аналогично, объект ServiceItem является экспертом по своей базовой стоимости и ограничениям (минимальное пожертвование, допустимое количество).

Ограничения:

- Проблемы в распределённой архитектуре: если данные о заказе и услугах хранятся или обрабатываются в разных сервисах, принцип Information Expert становится труднее применять, так как «эксперт» физически не владеет всей необходимой информацией в одном месте.

5.2.2. Расчёт запаса хода в бортовой системе автомобиля

В современной машине оценка запаса хода должна учитывать уровень топлива или заряда батареи, текущий расход энергии и режим движения. В соответствии с принципом Information Expert ответственность за этот расчёт возлагается на объект BatteryManagementUnit или PowertrainController, так как именно он располагает данными о доступной энергии и параметрах расхода. Объект ClimateControl, в свою очередь, является экспертом по собственному энергопотреблению и может предоставлять вклад в общий расход. Такое распределение обязанностей размещает вычисления рядом с источниками данных и делает модель более логичной.

Ограничения:

- Риск рассогласования данных: данные от разных датчиков могут обновляться с разной частотой, что приводит к временно неточным оценкам запаса хода.
- Зависимость от аппаратной платформы: логика, размещённая в экспертных модулях, становится тесно связанной с конкретными датчиками и контроллерами, что усложняет тестирование и переносимость.

5.3. Общие ограничения Information Expert

1. **Риск перегрузки доменных объектов:** Назначение ответственности объектам, владеющим наибольшим объёмом данных, может привести к концентрации в них значительной части бизнес-логики. В результате такие объекты становятся чрезмерно сложными и нарушают принцип единой ответственности.
2. **Сложность повторного использования логики:** Логика, размещённая внутри объектов-экспертов, может быть жёстко привязана к их внутреннему состоянию. Это затрудняет повторное использование вычислений и правил в других контекстах без дополнительной абстракции.
3. **Проблемы в распределённых системах:** В системах, где необходимые данные распределены между разными сервисами или компонентами, принцип Information Expert становится труднореализуемым, так как ни один объект не владеет всей требуемой информацией.

6. ВЫВОД

В ходе выполнения лабораторной работы были рассмотрены шаблоны проектирования из каталога GoF и принципы GRASP, а именно Bridge, Singleton, Creator и Information Expert. Для каждого из них были изучены назначение и основные идеи, предложены примеры сценариев использования, а также проанализированы возможные ограничения их применения. В результате было показано, что шаблоны проектирования и принципы GRASP позволяют снизить связанность компонентов, более обоснованно распределять обязанности между объектами и упростить развитие архитектуры системы. В то же время было выявлено, что неуместное или чрезмерное использование этих подходов может привести к избыточной сложности, ухудшению тестируемости и снижению гибкости системы. Таким образом, шаблоны проектирования и принципы GRASP являются полезными инструментами при проектировании программных систем, однако их применение должно основываться на анализе конкретного контекста и требований к сопровождаемости и масштабируемости.

.